

Reto: Bloque 2 día 10

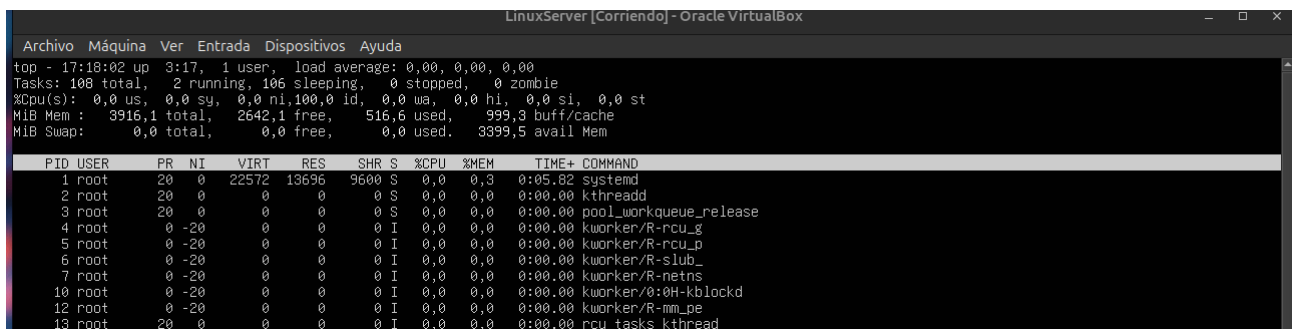
Fase: 1

En este apartado vamos a ejecutar y analizar las herramientas: top, htop, uptime y free. Después identificaremos el proceso que consume más CPU, más RAM y el tiempo de uso del sistema y la carga promedio.

Vamos por a empezar por analizar qué cada cada uno de los comandos.

- top: muestra la lista de procesos de forma dinámica.
- htop: es una versión más visual del comando top.
- uptime: muestra el tiempo de encendido y la carga media del sistema
- free: muestra la cantidad de RAM y de la partición SWAP.

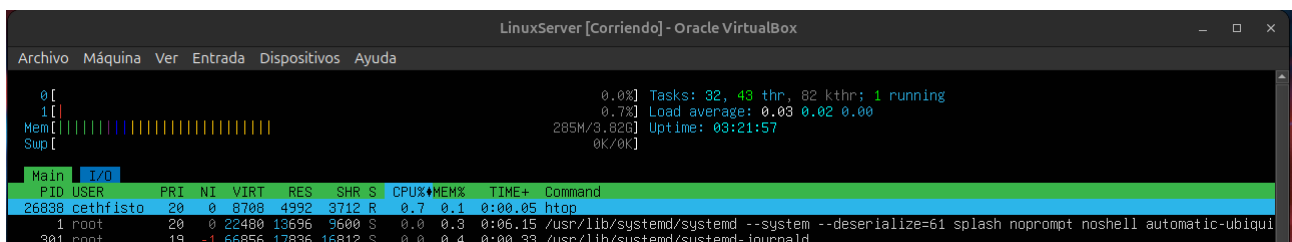
En esta imagen tenemos una captura de comando top, podemos ver que tenemos un 0% de uso de CPU y un proceso llamado systemd que consume 0,3% de memoria.



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
top - 17:18:02 up 3:17, 1 user, load average: 0,00, 0,00, 0,00
Tasks: 108 total, 2 running, 106 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0,0 us, 0,0 sy, 0,0 ni,100,0 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
MiB Mem : 3916,1 total, 2642,1 free, 516,6 used, 999,3 buff/cache
MiB Swap: 0,0 total, 0,0 free, 0,0 used, 3399,5 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   22572  13696  9600  S   0,0   0,3   0:05.82 systemd
    2 root        20   0         0      0      0  S   0,0   0,0   0:00.00 kthreadd
    3 root        20   0         0      0      0  S   0,0   0,0   0:00.00 pool_workqueue_release
    4 root         0 -20      0      0      0  I   0,0   0,0   0:00.00 kworker/R-rcu_g
    5 root         0 -20      0      0      0  I   0,0   0,0   0:00.00 kworker/R-rcu_p
    6 root         0 -20      0      0      0  I   0,0   0,0   0:00.00 kworker/R-slub_
    7 root         0 -20      0      0      0  I   0,0   0,0   0:00.00 kworker/R-netns
   10 root         0 -20      0      0      0  I   0,0   0,0   0:00.00 kworker/0:0H-kblockd
   12 root         0 -20      0      0      0  I   0,0   0,0   0:00.00 kworker/R-mm_pe
   13 root        20   0         0      0      0  I   0,0   0,0   0:00.00 rcu_tasks_kthread
```

En la captura estamos usando htop, podemos apreciar que es más visual que top. En esta caso, he conseguido hacer una captura donde el propio htop, es el proceso que más CPU usa con un 0,7%.

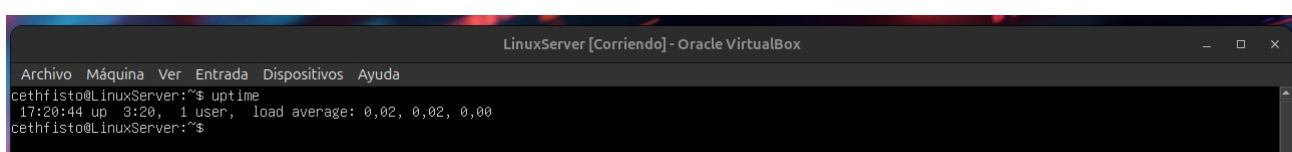


```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
0[                                     0.0%] Tasks: 32, 43 thr, 82 kthr; 1 running
1[                                     0.7%] Load average: 0.03 0.02 0.00
Mem[|||||] 285M/3.82G Uptime: 03:21:57
Swp[|||||] 0K/0K

Main I/O
  PID USER      PRI  NI   VIRT   RES   SHR  S  CPU% MEM%    TIME+  Command
26838 cethfisto  20   0   8708   4992  3712  R   0.7   0.1   0:00.05 htop
    1 root        20   0   22480  13696  9600  S   0,0   0,3   0:06.15 /usr/lib/systemd/systemd --system --deserialize=61 splash noprompt noshell automatic-ubiqui
 301 root        19  -1  66856  17836  16812  S   0,0   0,4   0:00.33 /usr/lib/systemd/systemd-journald
```

*Cabe destacar que tanto en top y htop los procesos que más consumen se ponen al principio del listado, para que sean más fáciles de identificar.

En esta captura tenemos el uso del comando uptime que podemos apreciar que la VM lleva encendida desde las 17:20 (3,2 horas) y una carga media de 0,02.



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
cethfisto@LinuxServer:~$ uptime
17:20:44 up 3:20, 1 user, load average: 0,02, 0,02, 0,00
cethfisto@LinuxServer:~$
```

En esta última captura tenemos el comando free, que nos informa de que tenemos 3.82 Gbs de memoria total, 0,51Gbs de memoria usada, 2,57 Gbs de memoria libre, 3,4Mb de memoria compartida, 0,98 memoria usada por el kernel y 3,31 Gbs de memoria disponible. Respecto al SWAP no la tenemos configurada por eso nos sale valor 0 en todos los campos.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
cethfisto@LinuxServer:~$ free
              total        used        free      shared  buff/cache   available
Mem:           4010072       536948       2695284         3480       1025988       3473124
Swap:              0              0              0
```

Fase: 2

Aquí, finalizaremos un proceso inactivo y no esencial con kill, killall o pkill. Cambiaremos la prioridad de un proceso en ejecución con renice. Lanzaremos un proceso en segundo plano y lo enviaremos al primer plano con fg. Y usaremos nice para iniciar un proceso de baja prioridad.

Para poder poder usar el comando kill tendremos que conocer el PID, en este ejercicio he finalizado el proceso 984.

```
984 cethfisto 20 0 21144 8516 1792 S 0.0 0.1 0:00.00 (sd-pam)
992 cethfisto 20 0 8764 5504 3840 S 0.0 0.1 0:00.01 -bash
3519 syslog 20 0 217M 5248 4480 S 0.0 0.1 0:00.02 /usr/sbin/rsyslogd -n -iNONE
3521 syslog 20 0 217M 5248 4480 S 0.0 0.1 0:00.02 /usr/sbin/rsyslogd -n -iNONE
3522 syslog 20 0 217M 5248 4480 S 0.0 0.1 0:00.00 /usr/sbin/rsyslogd -n -iNONE
3523 syslog 20 0 217M 5248 4480 S 0.0 0.1 0:00.01 /usr/sbin/rsyslogd -n -iNONE
9390 root 20 0 645M 42072 36392 S 0.0 1.0 0:01.10 /usr/libexec/fwupd/fwupd
9391 root 20 0 645M 42072 36392 S 0.0 1.0 0:00.20 /usr/libexec/fwupd/fwupd
9393 root 20 0 645M 42072 36392 S 0.0 1.0 0:00.00 /usr/libexec/fwupd/fwupd
9394 root 20 0 645M 42072 36392 S 0.0 1.0 0:00.20 /usr/libexec/fwupd/fwupd
9395 root 20 0 645M 42072 36392 S 0.0 1.0 0:00.00 /usr/libexec/fwupd/fwupd
9397 root 20 0 645M 42072 36392 S 0.0 1.0 0:00.00 /usr/libexec/fwupd/fwupd
9399 root 20 0 306M 9088 7552 S 0.0 0.2 0:00.05 /usr/libexec/upowerd
9401 root 20 0 306M 9088 7552 S 0.0 0.2 0:00.00 /usr/libexec/upowerd
9402 root 20 0 306M 9088 7552 S 0.0 0.2 0:00.00 /usr/libexec/upowerd
9403 root 20 0 306M 9088 7552 S 0.0 0.2 0:00.00 /usr/libexec/upowerd
9632 root 20 0 81380 2960 2688 S 0.0 0.1 0:00.00 gpg-agent --homedir /var/lib/fwupd/gnupg --use-standard-socket --daemon
14591 root 20 0 457M 13568 11392 S 0.0 0.3 0:00.05 /usr/libexec/udisks2/udisksd
14592 root 20 0 457M 13568 11392 S 0.0 0.3 0:00.15 /usr/libexec/udisks2/udisksd
cethfisto@LinuxServer:~$
cethfisto@LinuxServer:~$ kill 984
cethfisto@LinuxServer:~$ kill 984
-bash: kill: (984) - No such process
```

*En el caso de killall y pkill necesitaremos conocer el nombre del proceso. Por ejemplo killall/pkill nano.

Ahora, para cambiar la prioridad de un proceso usaremos el comando renice. Cambiare la prioridad del proceso 302 que tiene prioridad 19 a prioridad 18. Por ejemplo: sudo renice -n 18 -p 302.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
0% 0.0% Tasks: 29, 40 thr, 80 kthr: 2 running
0.0% 0.0% Load average: 0.02 0.10 0.07
241M/3.82G Uptime: 00:07:13
0K/0K
Main 1/0
PID USER PRI NI VIRT RES SHR S CPU%MEM% TIME+ Command
1133 cethfisto 20 0 8692 4992 3712 R 0.0 0.1 0:00.30 htop
1 root 20 0 22136 13156 9444 S 0.0 0.3 0:00.61 /sbin/init splash noprompt noshell automatic-ubuntu
302 root 19 -1 50440 15496 14472 S 0.0 0.4 0:00.09 /usr/lib/systemd/systemd-journald
355 root RT 0 282M 27136 8704 S 0.0 0.7 0:00.02 /sbin/multipathd -d -s
```

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

0% Tasks: 31, 50 thr, 79 kthr: 1 running
1% Load average: 0.00 0.06 0.05
Mem: 235M/3.82G Uptime: 00:09:23
Swap: 0K/0K

Main I/O
PID USER PRI NI UIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 22136 13156 9444 S 0.0 0.3 0:00.62 /sbin/init splash noprompt noshell automatic-ubiquity
302 root 38 18 50440 15624 14600 S 0.0 0.4 0:00.10 /usr/lib/systemd/systemd-journald
356 root RT 0 282M 27136 8704 S 0.0 0.7 0:00.02 /sbin/multipathd -d -s
365 root 20 0 29468 7936 4864 S 0.0 0.2 0:00.07 /usr/lib/systemd/systemd-udevd
```

* Como podemos ver con el htop el proceso ha cambiado de prioridad 19 a prioridad 18.

Para poner un proceso con &, usaremos el comando sleep. Por ejemplo: sleep 120 &. Y luego, para llevarlo otra vez al primer plano usaremos el fg. Por ejemplo fg %1.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

cethfisto@LinuxServer:~$ sleep 120 &
[1] 1226
cethfisto@LinuxServer:~$ fg %1
sleep 120
^C
cethfisto@LinuxServer:~$ _
```

* El comando sleep 120 &, indica que durante 120 segundos el proceso se para y el comando se ejecuta en segundo plano. Y con el comando fg %1 reinicia un proceso parado o lleva un proceso en segundo plano a primer plano.

Para iniciar un proceso con baja prioridad usaremos el comando nice.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

cethfisto@LinuxServer:~$ nice -n 10 cp archivogrande /ruta/destino_
```

* He puesto este ejemplo no funcional para indicar y explicar que este proceso se ejecutará en prioridad 10 para copiar un archivo grande a la ruta indicada.

Fase: 3

En esta fase, usaremos el comando vmstat para guardar su salida en un archivo /srv/logs/vmstat.log. Configuraremos una tarea en crontab que guarde el uso de recursos cada 5 minutos en /srv/logs/top.log. Y exploraremos con iotop para monitorizar el I/O del HDD si el sistema lo permite.

Para usar vmstat y guardara su salida en /srv/logs, crearemos el directorio y asignaremos el propietario.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

cethfisto@LinuxServer:~$ sudo mkdir -p /srv/logs
[sudo] password for cethfisto:
cethfisto@LinuxServer:~$ sudo chown $USER:$USER /srv/logs
cethfisto@LinuxServer:~$ _
```

Luego ejecutaremos vmstat y guardaremos la salida en el directorio creado.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
cethfisto@LinuxServer:~$ vmstat > /srv/logs/vmstat.log
cethfisto@LinuxServer:~$
```

Ahora configuraremos el crontab que guarde los recursos cada 5 minutos en /srv/logs/top.log. Ejecutaremos el cron con, crontab -e para editar el archivo de cron.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
cethfisto@LinuxServer:~$ crontab -e_
```

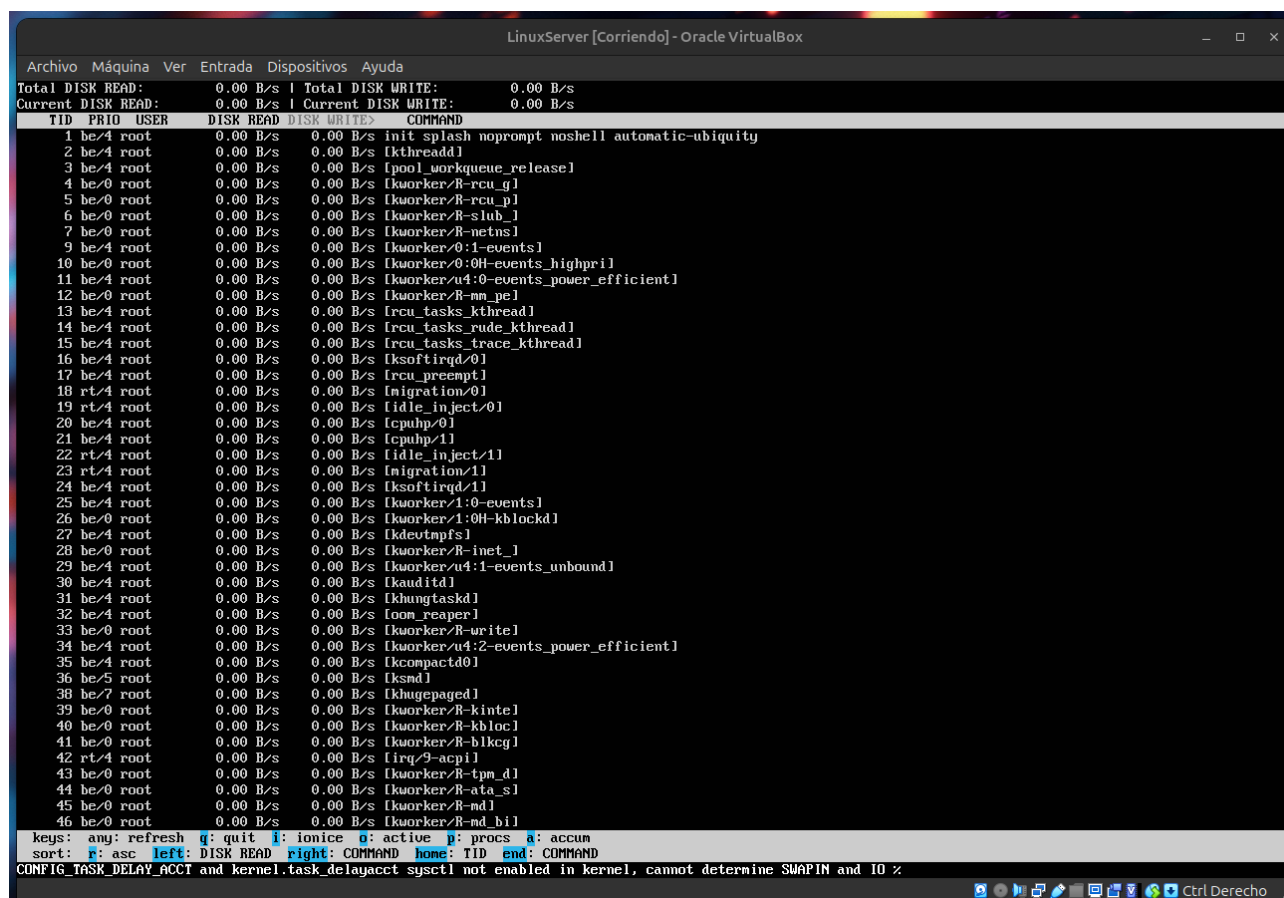
Añadiremos el comando al final del documento.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 7.2 /tmp/crontab.P9hwRj/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
*/5 * * * * top -b -n 1 >> /srv/logs/top.log
```

Y una vez guardado y terminado la configuración, podemos ver los logs creados con el comando tail /srv/logs/top.log.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
cethfisto@LinuxServer:~$ tail /srv/logs/top.log
1059 root      20  0    6940    4608    3968 S   0,0  0,0  0:00.01 login
1186 root      -2  0      0      0      0 S   0,0  0,0  0:00.00 psimon
1189 cethfis+   20  0    20164   11264   9344 S   0,0  0,3  0:00.08 systemd
1191 cethfis+   20  0    21148    3516   1792 S   0,0  0,1  0:00.00 (sd-pan)
1199 cethfis+   20  0    8656    5504    3840 S   0,0  0,1  0:00.02 bash
1245 root       0 -20    0      0      0 I   0,0  0,0  0:00.00 kworker+
1388 root      20  0    9532    3720    3200 S   0,0  0,1  0:00.00 cron
1389 cethfis+   20  0    2800    1536    1536 S   0,0  0,0  0:00.00 sh
1390 cethfis+   20  0   11928    5504    3456 R   0,0  0,1  0:00.00 top
1392 root      20  0      0      0      0 I   0,0  0,0  0:00.00 kworker+
cethfisto@LinuxServer:~$
```

Para poder usar iotop lo instalaremos con el comando `sudo apt install iotop`. Luego lo ejecutaremos con `sudo iotop`. Podremos ver todos los procesos de lectura-escritura (I/O) en nuestro HHD.



The screenshot shows the output of the `iotop` command. At the top, it displays disk I/O statistics: Total DISK READ: 0.00 B/s, Total DISK WRITE: 0.00 B/s, Current DISK READ: 0.00 B/s, and Current DISK WRITE: 0.00 B/s. Below this is a table with columns: TID, PRIO, USER, DISK READ, DISK WRITE, and COMMAND. The table lists various system processes, including `init splash noprompt noshell automatic-ubiquity`, `[kthreadd]`, `[pool_workqueue_release]`, `[kworker/R-rcu_gp]`, `[kworker/R-rcu_pl]`, `[kworker/R-slub]`, `[kworker/R-netns]`, `[kworker/0:1-events]`, `[kworker/0:0H-events_highpri]`, `[kworker/u4:0-events_power_efficient]`, `[kworker/R-nm_pe]`, `[rcu_tasks_kthread]`, `[rcu_tasks_rude_kthread]`, `[rcu_tasks_trace_kthread]`, `[ksftirqd/0]`, `[rcu_preempt]`, `[migration/0]`, `[idle_inject/0]`, `[cpuhp/0]`, `[cpuhp/1]`, `[idle_inject/1]`, `[migration/1]`, `[ksftirqd/1]`, `[kworker/1:0-events]`, `[kworker/1:0H-kblockd]`, `[kdeutmpfs]`, `[kworker/R-inet_]`, `[kworker/u4:1-events_unbound]`, `[kauditd]`, `[khungtaskd]`, `[oom_reaper]`, `[kworker/R-urwite]`, `[kworker/u4:2-events_power_efficient]`, `[kcompactd0]`, `[ksmd]`, `[khugepaged]`, `[kworker/R-kintel]`, `[kworker/R-kbld]`, `[kworker/R-bldcg]`, `[irq/9-acpi]`, `[kworker/R-tpm_d]`, `[kworker/R-ata_sl]`, `[kworker/R-md]`, and `[kworker/R-md_bll]`. At the bottom, there are keys for navigation and a note about CONFIG_TASK_DELAY_ACCT and kernel.task_delayacct sysctl.

TID	PRIO	USER	DISK READ	DISK WRITE	COMMAND
1	be/4	root	0.00 B/s	0.00 B/s	init splash noprompt noshell automatic-ubiquity
2	be/4	root	0.00 B/s	0.00 B/s	[kthreadd]
3	be/4	root	0.00 B/s	0.00 B/s	[pool_workqueue_release]
4	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-rcu_gp]
5	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-rcu_pl]
6	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-slub]
7	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-netns]
9	be/4	root	0.00 B/s	0.00 B/s	[kworker/0:1-events]
10	be/0	root	0.00 B/s	0.00 B/s	[kworker/0:0H-events_highpri]
11	be/4	root	0.00 B/s	0.00 B/s	[kworker/u4:0-events_power_efficient]
12	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-nm_pe]
13	be/4	root	0.00 B/s	0.00 B/s	[rcu_tasks_kthread]
14	be/4	root	0.00 B/s	0.00 B/s	[rcu_tasks_rude_kthread]
15	be/4	root	0.00 B/s	0.00 B/s	[rcu_tasks_trace_kthread]
16	be/4	root	0.00 B/s	0.00 B/s	[ksftirqd/0]
17	be/4	root	0.00 B/s	0.00 B/s	[rcu_preempt]
18	rt/4	root	0.00 B/s	0.00 B/s	[migration/0]
19	rt/4	root	0.00 B/s	0.00 B/s	[idle_inject/0]
20	be/4	root	0.00 B/s	0.00 B/s	[cpuhp/0]
21	be/4	root	0.00 B/s	0.00 B/s	[cpuhp/1]
22	rt/4	root	0.00 B/s	0.00 B/s	[idle_inject/1]
23	rt/4	root	0.00 B/s	0.00 B/s	[migration/1]
24	be/4	root	0.00 B/s	0.00 B/s	[ksftirqd/1]
25	be/4	root	0.00 B/s	0.00 B/s	[kworker/1:0-events]
26	be/0	root	0.00 B/s	0.00 B/s	[kworker/1:0H-kblockd]
27	be/4	root	0.00 B/s	0.00 B/s	[kdeutmpfs]
28	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-inet_]
29	be/4	root	0.00 B/s	0.00 B/s	[kworker/u4:1-events_unbound]
30	be/4	root	0.00 B/s	0.00 B/s	[kauditd]
31	be/4	root	0.00 B/s	0.00 B/s	[khungtaskd]
32	be/4	root	0.00 B/s	0.00 B/s	[oom_reaper]
33	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-urwite]
34	be/4	root	0.00 B/s	0.00 B/s	[kworker/u4:2-events_power_efficient]
35	be/4	root	0.00 B/s	0.00 B/s	[kcompactd0]
36	be/5	root	0.00 B/s	0.00 B/s	[ksmd]
38	be/7	root	0.00 B/s	0.00 B/s	[khugepaged]
39	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-kintel]
40	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-kbld]
41	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-bldcg]
42	rt/4	root	0.00 B/s	0.00 B/s	[irq/9-acpi]
43	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-tpm_d]
44	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-ata_sl]
45	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-md]
46	be/0	root	0.00 B/s	0.00 B/s	[kworker/R-md_bll]

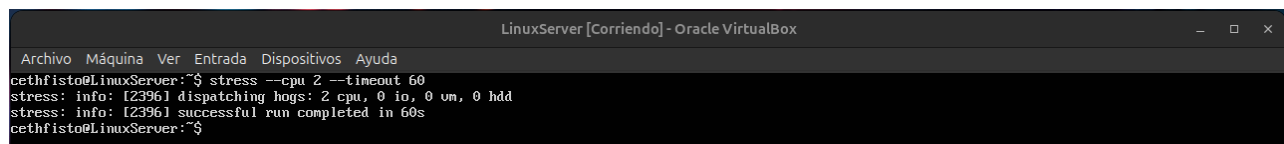
keys: any: refresh q: quit i: ionice o: active p: proc a: accum
sort: r: asc left: DISK READ right: COMMAND home: TID end: COMMAND
CONFIG_TASK_DELAY_ACCT and kernel.task_delayacct sysctl not enabled in kernel, cannot determine SWAPIN and IO %

Fase: 4

En este apartado, instalaremos el paquete `stress` o `stress-ng`. Ejecutaremos una prueba de carga de CPU, RAM y HHD durante 1 minuto. Y observaremos el comportamiento del sistema con `htop` y anotaremos el resultado.

Para tener el servicio de `stress` que se encarga de hacer pruebas de esfuerzo a nuestro equipo. Usaremos el comando `sudo apt install stress` o `sudo apt install stress-ng`.

Para la prueba de la CPU usaremos el comando `stress --cpu 2 --timeout 60`.



The screenshot shows a terminal window with the command `stress --cpu 2 --timeout 60` being executed. The output shows that the stress test is dispatching hogs for 2 CPU, 0 IO, 0 VM, and 0 HDD, and that the successful run completed in 60 seconds.

Command	Output
<code>cethfisto@LinuxServer:~\$ stress --cpu 2 --timeout 60</code>	<code>stress: info: [2396] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd</code>
<code>cethfisto@LinuxServer:~\$</code>	<code>stress: info: [2396] successful run completed in 60s</code>

* En nuestro caso hemos usado los 2 núcleos que tenía nuestra VM, pero podemos ajustarla a los núcleos que creamos conveniente.

Ahora hemos hecho la prueba a nuestra RAM, hemos usado el comando stress --vm 1 --vm-bytes 2G --timeout 60.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
cethfisto@LinuxServer:~$ stress --vm 1 --vm-bytes 2G --timeout 60
stress: info: [2406] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
stress: info: [2406] successful run completed in 60s
cethfisto@LinuxServer:~$
```

* En la captura hemos usado 2 gigabytes, pero podemos ajustar la cantidad según lo necesitemos.

Y para la prueba de HDD, hemos usado el comando stress --hdd 1 --hdd-bytes 1G --timeout 60

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
cethfisto@LinuxServer:~$ stress --hdd 1 --hdd-bytes 1G --timeout 60
stress: info: [2412] dispatching hogs: 0 cpu, 0 io, 0 vm, 1 hdd
stress: info: [2412] successful run completed in 60s
cethfisto@LinuxServer:~$
```

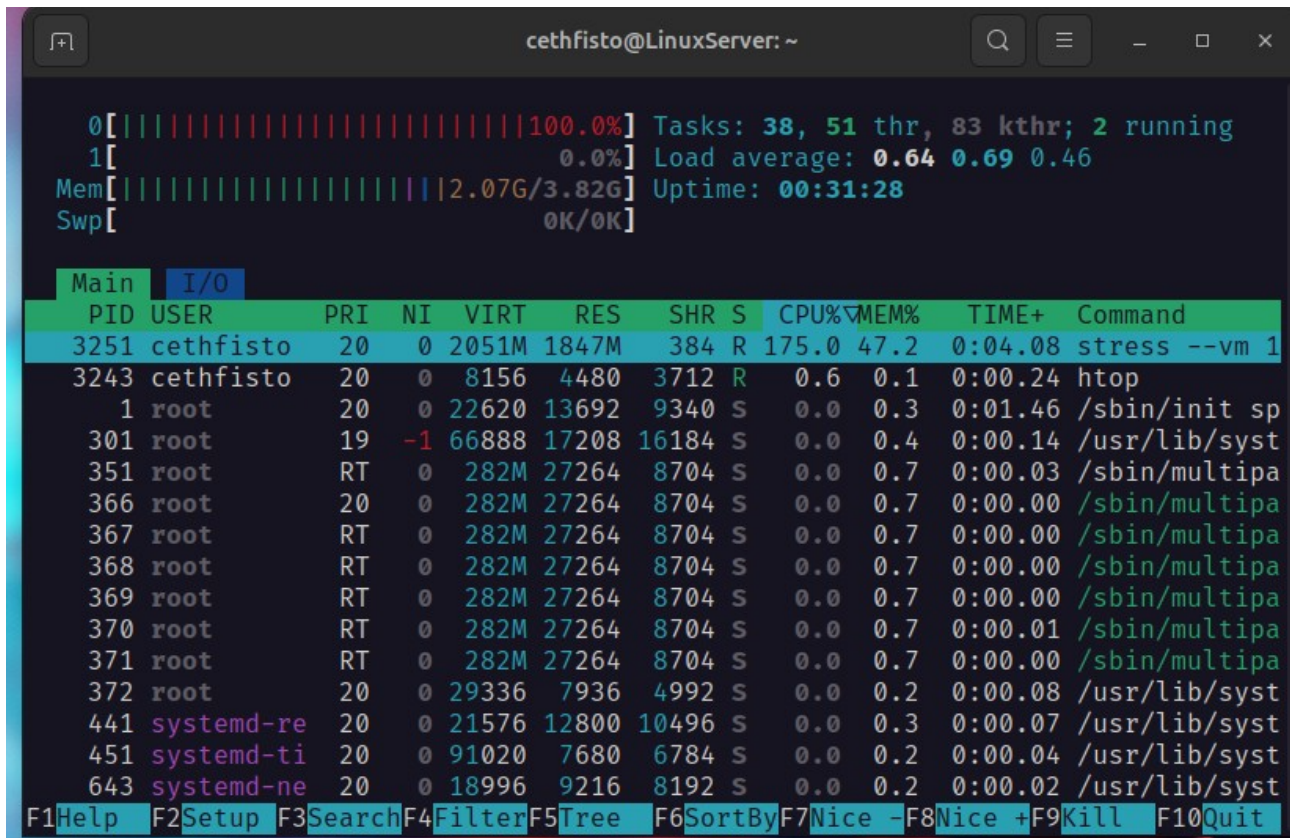
* En la captura hemos usado solo 1 gigabyte para hacer la prueba de carga al HDD de nuestra VM.

Para poder ver el comportamiento de la prueba de estrés con htop, primero deberemos hacer una conexión SSH a nuestra VM desde la máquina anfitrión. Después solo tenemos que ejecutar el comando htop mientras el proceso stress se ejecuta en nuestra vm.

```
cethfisto@LinuxServer: ~
0[|||||||||||||||||||||||||||||100.0%] Tasks: 39, 52 thr, 83 kthr; 2 running
1[|||||||||||||||||||||||||||||100.0%] Load average: 0.44 0.64 0.43
Mem[|||||] 300M/3.82G Uptime: 00:30:22
Swp[ ] 0K/0K

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
3246 cethfisto 20 0 3620 384 384 R 100.3 0.0 0:04.19 stress --cpu
3245 cethfisto 20 0 3620 384 384 R 99.7 0.0 0:04.19 stress --cpu
3243 cethfisto 20 0 8156 4480 3712 R 0.7 0.1 0:00.05 htop
1 root 20 0 22620 13692 9340 S 0.0 0.3 0:01.46 /sbin/init sp
301 root 19 -1 66888 17208 16184 S 0.0 0.4 0:00.14 /usr/lib/syst
351 root RT 0 282M 27264 8704 S 0.0 0.7 0:00.03 /sbin/multipa
366 root 20 0 282M 27264 8704 S 0.0 0.7 0:00.00 /sbin/multipa
367 root RT 0 282M 27264 8704 S 0.0 0.7 0:00.00 /sbin/multipa
368 root RT 0 282M 27264 8704 S 0.0 0.7 0:00.00 /sbin/multipa
369 root RT 0 282M 27264 8704 S 0.0 0.7 0:00.00 /sbin/multipa
370 root RT 0 282M 27264 8704 S 0.0 0.7 0:00.01 /sbin/multipa
371 root RT 0 282M 27264 8704 S 0.0 0.7 0:00.00 /sbin/multipa
372 root 20 0 29336 7936 4992 S 0.0 0.2 0:00.08 /usr/lib/syst
441 systemd-re 20 0 21576 12800 10496 S 0.0 0.3 0:00.07 /usr/lib/syst
451 systemd-ti 20 0 91020 7680 6784 S 0.0 0.2 0:00.04 /usr/lib/syst
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice -F8Nice +F9Kill F10Quit
```

* En esta captura se realizó cuando estábamos sometiendo la CPU a la prueba de estrés durante 1 minuto.



* Y esta captura fue hecha cuando estábamos sometiendo la RAM a la prueba de estrés usando como parámetro 2 gigabytes durante 1 minuto.

Fase: 5

En esta última fase vamos hacer un resumen rápido de cómo detectar problemas de rendimiento en Linux.

- Carga de CPU alta: deberemos mirar los procesos de más % de CPU y el uso constante de la CPU.
- Uso de la RAM alta: estaremos atento a que la RAM no esté cerca del 100%, uso excesivo de la partición SWAP y procesos que consumen mucha RAM.
- Problemas con el HDD: debemos localizar procesos con mucha escritura-lectura, particiones al 100% y archivos o carpetas grandes.
- Red lenta: estudiaremos si la tarjeta de red tiene tráfico alto o errores, conexiones sospechosas o pérdida de paquetes.